SERVER | WEEK 1

SQL入門
ゲームのデータを扱う

SQL

データベースにデータを入れ、取り出し、守る方法を、実際に書いて身につける

サーバー編 第1週（全3コマ） | 高等部3年

LESSON 1

データベースとSELECT

目標：テーブルを作り、欲しいデータだけを取り出せる

導入

ゲームのデータは、どこに保存されているか

手元（端末の中）

- Unity の PlayerPrefs、セーブファイル
- オフラインのゲームはこれで十分
- 遊ぶ人が**自由に書き換えられてしまう**
- スマホを壊すとデータも消える

サーバー（運営の建物の中）

- スマホゲーム、オンラインゲームはこちら
- 遊ぶ人は**直接さわれない**ので改造されにくい
- 機種変更しても引き継げる
- 全員のデータを並べて**ランキング**が作れる

サーバーで、何百万人分のデータを整理して保存しているのが**データベース**。それを操作する言葉が **SQL**。

POINT

2週間のゴール：**Unity のゲームから、サーバーのデータベースにスコアを保存し、オンラインランキングを表示する。**今週はその土台の SQL を書けるようになる。

導入

データベース・DBMS・SQL

言葉	意味	たとえば
データベース	整理して保存されたデータのかたまり	全プレイヤーのデータ
DBMS	データベースを管理するソフト	MySQL、PostgreSQL、SQLite
SQL	DBMS に命令を出すための言葉	SELECT name FROM players;

表で管理する

データを Excel のような**表**で持つ方式を「リレーショナルデータベース」と呼ぶ。今も一番使われている。

SQL は共通語

DBMS が違っても、基本の SQL はほぼ同じ。1つ覚えれば、どこでも使える。

この授業では SQLite

ファイル1つで動く小さな DBMS。スマホのアプリの中でもよく使われている。

POINT C# が「CPU に何をさせるか」を書く言葉なら、SQL は「**データベースから何が欲しいか**」を書く言葉。手順ではなく、欲しい結果を書く。

基本

テーブル・行・列

データベースの中には、**テーブル**（表）がいくつも入っている。下は players テーブル。

id	name	job	level	score	gems
1	アレン	剣士	52	98000	300
2	ミナ	魔法使い	47	120500	1200
3	ゴウ	戦士	60	87000	0

列（カラム）

データの項目。縦の並び。C# のクラスの**フィールド**にあたる。

行（レコード）

1件分のデータ。横の並び。C# の**インスタンス1つ**にあたる。

テーブル

同じ形の行を集めたもの。C# の **List<Player>** にあたる。

基本

SQL の命令は、ほぼこれだけ

命令	やること	ゲームでいうと
CREATE TABLE	テーブルを作る	セーブデータの形を決める
INSERT	行を追加する	新規登録、アイテム入手
SELECT	行を取り出す	ランキング表示、マイページ
UPDATE	行を書き換える	レベルアップ、ジェム消費
DELETE	行を消す	アイテムを売る、退会

文の終わりは ;

C# と同じ。1つの命令の終わりにセミコロン。

命令は大文字が習慣

select でも動くが、命令は大文字、名前は小文字にすると読みやすい。

文字列は ' '

'アレン' のようにシングルクォート。C# の " " とは違う。

準備

ブラウザだけで SQL を動かす

sqliteonline.com　　インストール・登録なし

1
ページを開く
左の一覧で SQLite が選ばれていることを確かめる

2
SQL を書く
中央の入力欄に書く。前の SQL は消してよい

3
Run を押す
上のボタン。結果は下に表で出る

4
エラーを読む
英語で出る。syntax error、no such column などの単語を見る

注意　データはブラウザの中にあるだけなので、**ページを閉じたり再読み込みしたりすると消える。**毎時間の最初に、配布する「準備用 SQL」を貼り付けて実行する。

POINT　消えるのは弱点でもあり、長所でもある。**壊しても、準備用 SQL をもう一度実行すれば最初の状態に戻せる**ので、怖がらずに試せる。

基本

列には「型」を決める

SQLite の型	入るもの	C# でいうと	players での例
INTEGER	整数	int、long	id、level、score、gems
REAL	小数	float、double	（命中率 0.85 など）
TEXT	文字列	string	name、job
BLOB	そのままのバイト列	byte[]	（画像など。あまり使わない）

NULL ＝ 値がない

C# の null と同じ。**0 や空文字とは別物**。「まだ決まっていない」「存在しない」を表す。

true / false がない

SQLite には bool の型がない。**1 と 0** の INTEGER で代わりにする。

注意

SQLite は型にゆるく、INTEGER の列に文字列を入れても**エラーにならない**ことがある。MySQL などは型が厳しい。型は「何を入れる列か」の宣言として、きちんと守る。

準備

まずは全文：players テーブルを作ってデータを入れる

```
CREATE TABLE players (
id INTEGER PRIMARY KEY,
name TEXT,
job TEXT,
level INTEGER,
score INTEGER,
gems INTEGER
);

INSERT INTO players (id, name, job, level, score, gems)
VALUES
(1, 'アレン', '剣士', 52, 98000, 300),
(2, 'ミナ', '魔法使い', 47, 120500, 1200),
(3, 'ゴウ', '戦士', 60, 87000, 0),
(4, 'リサ', '僧侶', 35, 45000, 50),
(5, 'ケン', '剣士', 18, 12000, 800),
(6, 'ユイ', '魔法使い', 55, 131000, 2500),
(7, 'ソラ', '盗賊', 41, 76000, 100),
(8, 'ハル', '剣士', 49, 99000, 0);
```

やること

配布の setup_players.sql を貼り付けて、Run で一度に実行する。

確かめる

SELECT * FROM players; を実行して、8人分出れば成功。

空白は自由

そろえるための空白や改行は、あってもなくても同じ。

CREATE TABLE

テーブルの形を決める

```
CREATE TABLE players (
id INTEGER PRIMARY KEY,
name TEXT,
job TEXT,
level INTEGER,
score INTEGER,
gems INTEGER
);
```

- **CREATE TABLE テーブル名 (…);** が形
- カッコの中に「列名 型」をカンマで区切って並べる
- 最後の列のあとには**カンマを付けない**（よくあるエラー）
- **PRIMARY KEY**（主キー）：行を見分けるための番号。同じ値は2つ入らない

なぜ名前ではなく id で見分けるのか

同じ名前のプレイヤーは何人もいるし、名前は途中で変えられる。だから、**重ならず、変わらない番号**を1人に1つ付けて、それで見分ける。

INSERT

行を追加する

```sql
-- 1人だけ追加する
INSERT INTO players (id, name, job, level, score, gems)
VALUES (9, 'ナギ', '弓使い', 1, 0, 100);

-- 何人もまとめて追加する（カンマでつなぐ）
VALUES (1, 'アレン', …), (2, 'ミナ', …), … ;
```

列と値は同じ順番

前のカッコの1つ目の列に、後ろのカッコの1つ目の値が入る。

書かなかった列は NULL

列を省略すると、その列は「値なし」になる。

列名は省略しない

省略もできるが、あとで列が増えたときに壊れる。必ず書く習慣を。

試そう

上の「ナギ」を追加し、SELECT * FROM players; で9人になったか確認する。確認できたら **DELETE FROM players WHERE id = 9;** で消しておく（意味は2時間目）。

SELECT

データを取り出す

```sql
-- 全部の列
SELECT * FROM players;
```

*（アスタリスク）は「全部の列」。中身をざっと見たいときに使う。

```sql
-- 欲しい列だけ
SELECT name, level FROM players;
```

name	level
アレン	52
ミナ	47
ゴウ	60
…（8人分）	

POINT 「**SELECT 何を FROM どこから**」と読む。本番のサーバーでは * を避け、必要な列だけ取り出す。送るデータが減って速くなる。

WHERE

条件に合う行だけ取り出す

```sql
SELECT name, level
FROM players
WHERE level ⩾ 50;
```

name	level
アレン	52
ゴウ	60
ユイ	55

SQL	C#	意味
=	==	等しい（= は1つ！）
<>	!=	等しくない
> >=	> >=	より大きい、以上
< <=	< <=	より小さい、以下

```sql
-- 文字列は ' ' で囲む
SELECT * FROM players WHERE job = '魔法使い'; -- ミナ、ユイ
```

演習 1-1

SELECT と WHERE に慣れよう

No.	問題
1	全員の名前とスコアを表示する
2	id が 4 のプレイヤーの、すべての列を表示する
3	スコアが 100000 以上のプレイヤーの名前とスコアを表示する
4	剣士の名前とレベルを表示する
5	ジェムが 500 より少ないプレイヤーの名前とジェムを表示する
6	職業が魔法使いではないプレイヤーの名前を表示する

ヒント　形はいつも **SELECT 列 FROM players WHERE 条件;**。文字列の条件は ' ' で囲む。「より少ない」は <、「以上」は >=。

演習 1-1 解答

答えと結果

No.	SQL	結果
1	SELECT name, score FROM players;	8人分
2	SELECT * FROM players WHERE id = 4;	リサ 僧侶 35 45000 50
3	SELECT name, score FROM players WHERE score >= 100000;	ミナ 120500、ユイ 131000
4	SELECT name, level FROM players WHERE job = '剣士';	アレン 52、ケン 18、ハル 49
5	SELECT name, gems FROM players WHERE gems < 500;	アレン、ゴウ、リサ、ソラ、ハル
6	SELECT name FROM players WHERE job <> '魔法使い';	ミナとユイ以外の6人

よくある間違い 100000 を 100,000 とカンマ付きで書く（数にカンマは付けない）。'剣士' のクォート忘れ。

WHERE

条件を組み合わせる

書き方	例（WHERE のあと）	結果
AND	job = '剣士' AND level >= 40	アレン、ハル
OR	level >= 55 OR gems >= 1000	ミナ、ゴウ、ユイ
BETWEEN	score BETWEEN 80000 AND 100000	アレン、ゴウ、ハル
IN	job IN ('僧侶', '盗賊')	リサ、ソラ
LIKE	name LIKE '%ン'	アレン、ケン
IS NULL	job IS NULL	（今はなし）

LIKE の %

「何文字でもよい」の意味。'%ン' はンで終わる、'ア%' はアで始まる、'%ル%' はルを含む。

NULL に = は使えない

job = NULL は何も返さない。NULL を調べるときは必ず **IS NULL**、**IS NOT NULL**。

ORDER BY ／ LIMIT

並べ替えて、上から何件か：ランキング

```sql
-- スコアの上位3人
SELECT name, score
FROM players
ORDER BY score DESC
LIMIT 3;
```

name	score
ユイ	131000
ミナ	120500
ハル	99000

書き方	意味
ASC	小さい順（書かなければこれ）
DESC	大きい順
LIMIT 3	先頭から3件だけ
OFFSET 3	先頭の3件を飛ばす

```sql
-- 4〜6位（ランキングの2ページ目）
… LIMIT 3 OFFSET 3;
-- アレン、ゴウ、ソラ
```

POINT ORDER BY を書かないと、**並び順は保証されない**。たまたま id 順に見えても、順番が必要なら必ず書く。

計算と AS

取り出しながら計算し、名前を付ける

```sql
SELECT name || ' (' || job || ') ' AS label,
score / 1000 AS score_k,
score / 1000.0 AS score_k2
FROM players WHERE id = 2;
```

label	score_k	score_k2
ミナ（魔法使い）	120	120.5

+ − * / が使える

列どうし、列と数で計算できる。元のデータは変わらない。

|| で文字をつなぐ

C# の + にあたる。SQL の + は数の足し算だけ。

整数 ÷ 整数は切り捨て

C# と同じ。120500 / 1000 は 120。小数が欲しければ 1000.0 で割る。

演習 1-2

players から取り出そう

No.	問題
1	全員の名前と職業を表示する
2	ジェムを1つも持っていないプレイヤーの名前を表示する
3	レベルが 40 以上 50 未満のプレイヤーの名前とレベルを表示する
4	剣士を、レベルの高い順に表示する
5	レベルの低い順に並べたとき、2番目と3番目のプレイヤーの名前とレベルを表示する
6	名前と「次のレベルまでの経験値（レベル × 100）」を next_exp という名前で表示する

ヒント　まず「どの列が欲しいか（SELECT）」、次に「どの行が欲しいか（WHERE）」、最後に「どう並べるか（ORDER BY、LIMIT）」の順に考える。

演習 1-2 解答

答えと結果

No.	SQL	結果
1	SELECT name, job FROM players;	8人分
2	SELECT name FROM players WHERE gems = 0;	ゴウ、ハル
3	SELECT name, level FROM players WHERE level BETWEEN 40 AND 49;	ミナ 47、ソラ 41、ハル 49
4	SELECT * FROM players WHERE job = '剣士' ORDER BY level DESC;	アレン 52、ハル 49、ケン 18
5	SELECT name, level FROM players ORDER BY level LIMIT 2 OFFSET 1;	リサ 35、ソラ 41
6	SELECT name, level * 100 AS next_exp FROM players;	アレン 5200、ミナ 4700 …

よくある間違い '剣士' のクォート忘れ（no such column: 剣士）、== と書く、ORDER BY を WHERE の前に書く、最後の ; 忘れ。

演習 1-3（チャレンジ）

組み合わせて取り出そう

No.	問題
1	名前が「ア」か「ユ」で始まるプレイヤーの名前を表示する
2	レベル 40 以上の剣士と戦士を、スコアの高い順に表示する
3	ジェムを一番多く持っているプレイヤーの名前とジェムを表示する
4	スコアを1万点単位で切り捨てた値（98000 → 90000）を score_floor という名前で表示する
5	「アレン Lv.52」のような文字を、全員分 label という名前で表示する
6	レベルの高い順に並べたとき、6位から8位のプレイヤーを表示する

ヒント　3 は「並べて1件だけ」。4 は整数の割り算が切り捨てになることを使う。5 は || で文字をつなぐ。

演習 1-3 解答

答えと結果

```sql
-- 1 → アレン、ユイ
SELECT name FROM players WHERE name LIKE 'ア%' OR name LIKE 'ユ%';
-- 2 → ハル 99000、アレン 98000、ゴウ 87000
SELECT name, job, level, score FROM players
WHERE job IN ('剣士', '戦士') AND level ≥ 40
ORDER BY score DESC;
-- 3 → ユイ 2500
SELECT name, gems FROM players ORDER BY gems DESC LIMIT 1;
-- 4 → アレン 90000、ミナ 120000、ゴウ 80000 …
SELECT name, score / 10000 * 10000 AS score_floor FROM players;
-- 5 → アレン Lv.52、ミナ Lv.47 …
SELECT name || ' Lv.' || level AS label FROM players;
-- 6 → ソラ 41、リサ 35、ケン 18
SELECT name, level FROM players ORDER BY level DESC LIMIT 3 OFFSET 5;
```

LESSON 2

書き換え・集計・テーブルをつなぐ

目標：データを安全に書き換え、複数のテーブルから必要な情報を組み立てられる

準備

まずは players を作り直す

前の時間のデータは、ページを閉じた時点で消えている。SQL Online IDE（sqliteonline.com）を開き、**setup_players.sql** を貼り付けて Run する。

```
DROP TABLE IF EXISTS players; -- あれば消す（何度実行しても大丈夫にするため）
CREATE TABLE players ( … ); -- 1時間目と同じ
INSERT INTO players … ; -- 1時間目と同じ8人

-- 確かめる
SELECT * FROM players;
```

POINT この時間はデータを**書き換える**。間違えて壊しても、setup_players.sql をもう一度実行すれば最初の8人に戻る。

UPDATE

データを書き換える

```sql
-- ケンがレベルアップ
UPDATE players
SET level = level + 1
WHERE id = 5;

-- 2つの列を一度に変える
UPDATE players
SET level = 1, score = 0
WHERE id = 5;
```

- **UPDATE どこを SET 何をどう WHERE どの行**
- level = level + 1 は C# の level += 1 と同じ考え方
- 変える列はカンマで並べる
- WHERE では **id**（主キー）で行を指定するのが基本

試そう 1つ目を実行して、SELECT * FROM players WHERE id = 5; で確かめる。確かめたら **level = level - 1** で元に戻す（2つ目は例なので実行しない）。

UPDATE の落とし穴

WHERE を忘れると、全員が書き換わる

```
-- ケンのジェムを 0 にするつもりが…
UPDATE players
SET gems = 0;
```

エラーは出ない。**8人全員**のジェムが 0 になり、「8行が変更されました」と表示されるだけ。

本番のサーバーなら

全ユーザー

の課金アイテムが消える。
お詫びとニュースになる事故

防ぎ方	やること
先に SELECT で確かめる	同じ WHERE で SELECT して、変わる行が正しいか見てから UPDATE に書き換える
WHERE から書く	UPDATE を書くときは、SET より先に WHERE を書く習慣をつける
バックアップ	本番のデータを手で直す前には、必ず複製を取っておく

DELETE

行を消す、そして「本当は消さない」

```sql
DELETE FROM players WHERE id = 9; -- 行ごと消える。WHERE を忘れると全員消える
```

物理削除（DELETE）

- データが本当に消える
- 「間違えて退会した」と言われても戻せない
- 過去の課金の記録なども一緒に失う

論理削除（印を付ける）

- is_deleted という列を用意し、退会したら 1 にする
- SELECT するときは WHERE is_deleted = 0
- ゲームでは**こちらがよく使われる**

POINT 論理削除は、DELETE ではなく **UPDATE players SET is_deleted = 1 WHERE id = 5;** になる。「消す」も書き換えの一種として扱う。

演習 2-1

ゲーム運営の作業をやってみよう

No.	問題（上から順番に実行する）
1	不具合の補償：ソラ（id 7）のスコアを 80000 にする
2	リサ（id 4）のレベルを 5 上げる
3	イベントボーナス：全員のスコアを 1000 点増やす
4	レベルが 20 未満のプレイヤーを削除する
5	魔法使いのジェムを半分にする

ルール

書き換える前に、**同じ WHERE で SELECT して**変わる行を確かめる。全部終わったら SELECT * FROM players; で結果を見てから、**setup_players.sql で元に戻す。**

演習 2-1 解答

答えと結果

```sql
-- 1
UPDATE players SET score = 80000 WHERE id = 7;
-- 2 → リサのレベルは 40
UPDATE players SET level = level + 5 WHERE id = 4;
-- 3 → 8行が変わる
UPDATE players SET score = score + 1000;
-- 4 → ケンだけが消える
DELETE FROM players WHERE level < 20;
-- 5 → ミナ 600、ユイ 1250
UPDATE players SET gems = gems / 2 WHERE job = '魔法使い';
```

name	level	score	gems
アレン	52	99000	300
ミナ	47	121500	600
ゴウ	60	88000	0
リサ	40	46000	50
ユイ	55	132000	1250
ソラ	41	81000	100
ハル	49	100000	0

忘れずに 確かめたら **setup_players.sql を実行**して、最初の8人に戻してから次へ進む。

集計

何行もの値を、1つにまとめる

```sql
SELECT COUNT(*), SUM(gems), AVG(level), MAX(score), MIN(level)
FROM players;
```

COUNT(*)	SUM(gems)	AVG(level)	MAX(score)	MIN(level)
8	4950	44.625	131000	18
行の数	合計	平均	最大	最小

```sql
-- WHERE と組み合わせる：剣士は何人？
SELECT COUNT(*) FROM players WHERE job = '剣士'; -- 3
```

GROUP BY

グループごとに集計する

```sql
SELECT job,
COUNT(*) AS member_count,
ROUND(AVG(level), 1) AS avg_level
FROM players
GROUP BY job
ORDER BY avg_level DESC;
```

同じ job の行を1つの**グループ**にまとめ、グループごとに COUNT や AVG を計算する。
ROUND(値, 1) は小数第1位までに丸める。

job	member_count	avg_level
戦士	1	60.0
魔法使い	2	51.0
盗賊	1	41.0
剣士	3	39.7
僧侶	1	35.0

注意 GROUP BY を使うとき、SELECT に書けるのは**グループの列（job）と集計した値**だけ。name を書いても、グループの中の誰の名前が出るかは決まらない。

POINT 「職業ごとの人数」はゲームバランス調整の基本データ。剣士だけ多い、僧侶が育っていない、などが一目で分かる。

WHERE と HAVING は、絞り込むタイミングが違う

```sql
-- 2人以上いる職業だけ
SELECT job, COUNT(*) AS member_count
FROM players
GROUP BY job
HAVING COUNT(*) ⩾ 2;
-- 剣士 3、魔法使い 2
```

WHERE

グループに分ける**前**に、1行ずつ絞る

HAVING

グループに分けた**後**に、集計結果で絞る

実行順	句	書く順
1	FROM	2
2	WHERE	3
3	GROUP BY	4
4	HAVING	5
5	SELECT	1
6	ORDER BY	6
7	LIMIT	7

POINT

SQL は**書く順と実行される順が違う**。SELECT は書くのは最初だが、実行は後半。だから WHERE の中では、SELECT で付けた別名（AS）が使えない。

演習 2-2

集計しよう

No.	問題
1	全プレイヤーのジェムの合計と平均を、total_gems、avg_gems という名前で表示する
2	職業ごとの最高スコアを表示する
3	平均レベルが 45 以上の職業と、その平均レベルを表示する
4	スコアが 90000 以上のプレイヤーは何人いるか
5	職業ごとのジェムの合計を、多い順に表示する

ヒント 「〜ごと」と書いてあったら GROUP BY。集計した結果で絞るなら HAVING、集計の前に1行ずつ絞るなら WHERE。

演習 2-2 解答

答えと結果

```sql
-- 1 → 4950、618.75
SELECT SUM(gems) AS total_gems, AVG(gems) AS avg_gems FROM players;
-- 2 → 剣士 99000、魔法使い 131000、戦士 87000、僧侶 45000、盗賊 76000
SELECT job, MAX(score) AS best_score FROM players GROUP BY job;
-- 3 → 魔法使い 51.0、戦士 60.0
SELECT job, AVG(level) AS avg_level FROM players
GROUP BY job HAVING AVG(level) ⩾ 45;
-- 4 → 4（ユイ、ミナ、ハル、アレン）
SELECT COUNT(*) FROM players WHERE score ⩾ 90000;
-- 5 → 魔法使い 3700、剣士 1100、盗賊 100、僧侶 50、戦士 0
SELECT job, SUM(gems) AS total FROM players GROUP BY job ORDER BY total DESC;
```

よくある間違い 3 を WHERE AVG(level) >= 45 と書く（集計で絞るのは HAVING）。4 に GROUP BY を付けて、何も出なくなる・行が分かれる。

演習 2-3

運営チームが見たい数字を出そう

No.	問題
1	全員のスコアの平均を、小数第1位まで表示する
2	一番高いレベルと一番低いレベルの差を表示する
3	職業ごとの人数を、多い順に表示する
4	レベル 40 以上のプレイヤーだけで、職業ごとの平均スコアを表示する
5	1人しかいない職業の名前を表示する
6	ジェムの合計が 1000 以上の職業と、その合計を表示する

ヒント　2 は集計した値どうしで計算できる：MAX(level) - MIN(level)。4 は WHERE と GROUP BY、5 と 6 は HAVING。

演習 2-3 解答

答えと結果

```sql
-- 1 → 83562.5
SELECT ROUND(AVG(score), 1) AS avg_score FROM players;
-- 2 → 42（60 - 18）
SELECT MAX(level) - MIN(level) AS level_gap FROM players;
-- 3 → 剣士 3、魔法使い 2、ほかは 1
SELECT job, COUNT(*) AS member_count FROM players GROUP BY job ORDER BY member_count DESC;
-- 4 → 剣士 98500、魔法使い 125750、戦士 87000、盗賊 76000（僧侶は出ない）
SELECT job, AVG(score) AS avg_score FROM players WHERE level ⩾ 40 GROUP BY job;
-- 5 → 戦士、僧侶、盗賊
SELECT job FROM players GROUP BY job HAVING COUNT(*) = 1;
-- 6 → 魔法使い 3700、剣士 1100
SELECT job, SUM(gems) AS total FROM players GROUP BY job HAVING SUM(gems) ⩾ 1000;
```

POINT 4 で僧侶が出ないのは、リサ（レベル 35）が WHERE の時点で外れ、グループそのものができないから。

テーブル設計

持ち物も1つの表に書くと、何が困るか

name	job	item_name	rarity	count
アレン	剣士	やくそう	N	5
アレン	剣士	鉄の剣	R	1
アレン	剣士	伝説の剣	SSR	1
ミナ	魔法使い	薬草	N	3

同じことを何度も書く

アレンが転職したら、3行とも直す必要がある。1行直し忘れると、矛盾したデータになる。

表記がゆれる

「やくそう」と「薬草」が混ざると、「やくそうを持っている人」の集計がずれる。

アイテムの修正が大変

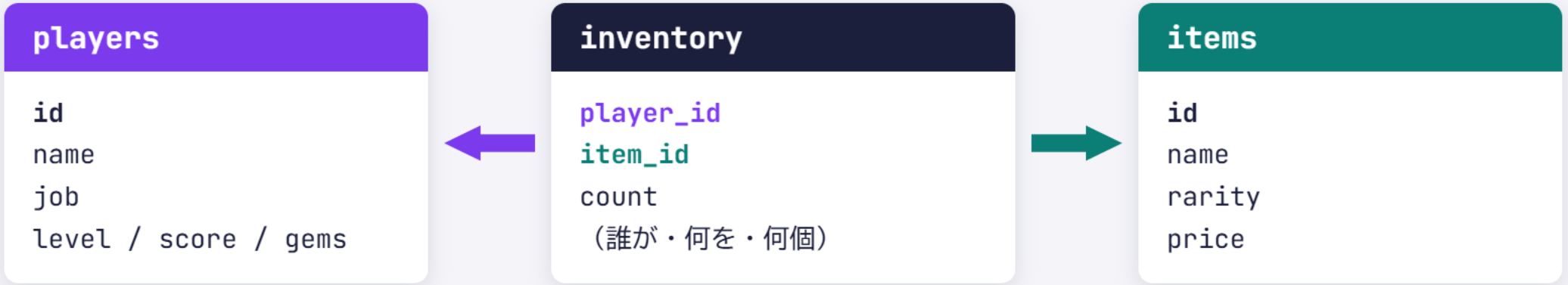
伝説の剣のレア度を変えたら、持っている全員の行を直すことになる。

何も持たない人が書けない

持ち物が0個のプレイヤーは、行が1つもなくなり、存在そのものが消える。

テーブル設計

表を分けて、id でつなぐ



1つの事実は1か所だけ

アイテムの名前もレア度も items に1回だけ。直すときも1か所。この整理を**正規化**と呼ぶ。

マスタとユーザーデータ

items は企画が決める**マスタデータ**。players と inventory は遊ぶ人ごとの**ユーザーデータ**。

POINT inventory には名前を書かず、**番号（id）だけ**を書く。名前が必要になったら、表をつなげて引いてくる。それが次の **JOIN**。

準備

まずは全文：items と inventory を作る

```sql
CREATE TABLE items (
id INTEGER PRIMARY KEY,
name TEXT,
rarity TEXT,
price INTEGER
);

CREATE TABLE inventory (
player_id INTEGER,
item_id INTEGER,
count INTEGER,
PRIMARY KEY (player_id, item_id)
);
```

```sql
INSERT INTO items (id, name, rarity, price)
VALUES
(1, 'やくそう', 'N', 10),
(2, '鉄の剣', 'R', 300),
(3, '魔法の杖', 'R', 350),
(4, 'ドラゴンの盾', 'SR', 1200),
(5, '伝説の剣', 'SSR', 5000);

INSERT INTO inventory (player_id, item_id, count)
VALUES
(1, 1, 5), (1, 2, 1), (1, 5, 1),
(2, 1, 3), (2, 3, 1), (3, 2, 1),
(3, 4, 1), (4, 1, 10), (6, 3, 1),
(6, 5, 1), (7, 1, 2);
```

POINT inventory の主キーは**2つの列の組み合わせ**。「同じ人が同じアイテムの行を2つ持つ」ことを防ぐ。ケン（5）とハル（8）は何も持っていない。

JOIN

2つの表をつなげて取り出す

```
-- アレン（id 1）の持ち物
SELECT items.name, inventory.count
FROM inventory
JOIN items
ON inventory.item_id = items.id
WHERE inventory.player_id = 1;
```

name	count
やくそう	5
鉄の剣	1
伝説の剣	1

- **JOIN つなぐ表 ON つなぐ条件**
- inventory の1行ごとに、item_id と同じ id を持つ items の行を探して、横にくっつける
- 両方の表に同じ列名（name、id）があるので、**表名.列名** と書いて区別する
- JOIN してできた大きな表に対して、WHERE や ORDER BY が使える

POINT inventory には番号しかないが、JOIN すれば名前もレア度も値段も引ける。**分けて保存し、つないで取り出す**のがリレーショナルデータベースの考え方。

JOIN

3つの表をつなぐ：誰が・何を持っているか

```sql
-- レア度 SR 以上を持っている人
SELECT p.name AS player_name, i.name AS item_name, i.rarity
FROM inventory AS inv
JOIN players AS p ON inv.player_id = p.id
JOIN items AS i ON inv.item_id = i.id
WHERE i.rarity IN ('SR', 'SSR')
ORDER BY p.id;
```

player_name	item_name	rarity
アレン	伝説の剣	SSR
ゴウ	ドラゴンの盾	SR
ユイ	伝説の剣	SSR

- JOIN はいくつでも続けて書ける
- **AS p** で表に短い別名を付けると、p.name のように短く書ける
- 別名を付けたら、その文の中では別名で呼ぶ

POINT 真ん中の inventory を中心に、左右の表を引き寄せるイメージ。inventory のような「つなぐための表」は、多対多（1人が何種類も、1種類を何人も）を表すときに必ず出てくる。

LEFT JOIN

相手がいない行も残す

```sql
-- 全員の「持っているアイテムの種類数」
SELECT p.name, COUNT(inv.item_id) AS item_kinds
FROM players AS p
LEFT JOIN inventory AS inv
ON p.id = inv.player_id
GROUP BY p.id, p.name
ORDER BY p.id;
```

JOIN

両方にある行だけ。ケンとハルは**消える**

LEFT JOIN

左の表は全部残す。相手がなければ NULL

name	item_kinds
アレン	3
ミナ	2
ゴウ	2
リサ	1
ケン	0
ユイ	2
ソラ	1
ハル	0

POINT 「全員の一覧」を作るときは LEFT JOIN。JOIN にすると、何も持っていない人が**ランキングや一覧から黙って消える**バグになる。

演習 2-4

表をつないで答えよう

No.	問題
1	ミナ（id 2）が持っているアイテムの名前と個数を表示する
2	「伝説の剣」を持っているプレイヤーの名前を表示する
3	プレイヤーごとに、持ち物の合計金額（値段 × 個数）を高い順に表示する
4	全プレイヤーの名前と、持っているアイテムの合計個数を表示する（何も持っていない人は 0）

ヒント

3 は JOIN してから GROUP BY。SUM の中に計算式を書ける：SUM(i.price * inv.count)。
4 は LEFT JOIN。SUM は相手がいないと NULL になるので、**COALESCE(値, 0)**（NULL なら 0 にする）で包む。

演習 2-4 解答

答えと結果

```
-- 1 → やくそう 3、魔法の杖 1
SELECT i.name, inv.count
FROM inventory AS inv
JOIN items AS i ON inv.item_id = i.id
WHERE inv.player_id = 2;

-- 2 → アレン、ユイ
SELECT p.name
FROM inventory AS inv
JOIN players AS p ON inv.player_id = p.id
JOIN items AS i ON inv.item_id = i.id
WHERE i.name = '伝説の剣';
```

```
-- 3 → アレン 5350、ユイ 5350、ゴウ 1500、
-- ミナ 380、リサ 100、ソラ 20
SELECT p.name, SUM(i.price * inv.count) AS total
FROM inventory AS inv
JOIN players AS p ON inv.player_id = p.id
JOIN items AS i ON inv.item_id = i.id
GROUP BY p.id, p.name
ORDER BY total DESC;

-- 4 → アレン 7、ミナ 4、ゴウ 2、リサ 10、
-- ケン 0、ユイ 2、ソラ 2、ハル 0
SELECT p.name, COALESCE(SUM(inv.count), 0) AS total_count
FROM players AS p
LEFT JOIN inventory AS inv ON p.id = inv.player_id
GROUP BY p.id, p.name;
```

演習 2-5

もっと表をつなごう

No.	問題
1	アレン（id 1）の持ち物の名前と値段を、値段の高い順に表示する
2	やくそう（id 1）を持っているプレイヤーの名前と個数を、多い順に表示する
3	レア度ごとに、持っている人の数（のべ人数）を表示する
4	何も持っていないプレイヤーの名前を表示する
5	だれも持っていないアイテムの名前を表示する

ヒント 4 と 5 は LEFT JOIN して、**相手が NULL の行だけ**を WHERE … IS NULL で残す。5 は items を左に置く。

演習 2-5 解答

答えと結果

```sql
-- 1 → 伝説の剣 5000、鉄の剣 300、やくそう 10
SELECT i.name, i.price FROM inventory AS inv
JOIN items AS i ON inv.item_id = i.id
WHERE inv.player_id = 1 ORDER BY i.price DESC;

-- 2 → リサ 10、アレン 5、ミナ 3、ソラ 2
SELECT p.name, inv.count FROM inventory AS inv
JOIN players AS p ON inv.player_id = p.id
WHERE inv.item_id = 1 ORDER BY inv.count DESC;

-- 3 → N 4、R 4、SR 1、SSR 2
SELECT i.rarity, COUNT(*) AS owners
FROM inventory AS inv
JOIN items AS i ON inv.item_id = i.id
GROUP BY i.rarity;
```

```sql
-- 4 → ケン、ハル
SELECT p.name
FROM players AS p
LEFT JOIN inventory AS inv
ON p.id = inv.player_id
WHERE inv.player_id IS NULL;

-- 5 → 0件
--（全部だれかが持っている）
SELECT i.name
FROM items AS i
LEFT JOIN inventory AS inv
ON i.id = inv.item_id
WHERE inv.item_id IS NULL;
```

LESSON 3

データを守る仕組みと総合演習

目標：おかしなデータや中途半端な処理を防ぐ書き方を使い、ゲームの画面に必要な SQL を書ける

準備

3つのテーブルを作り直す

SQL Online IDE を開き、**setup_game.sql** を貼り付けて Run する。players・items・inventory が2時間目の最初の状態でそろう。

準備用 SQL	作るテーブル	使うとき
setup_players.sql	players	1・2時間目
setup_game.sql	players、items、inventory	3時間目の最初
setup_final.sql	上の3つ ＋ accounts、gacha_logs	総合演習の前

```
-- 確かめる：8、5、11 と出れば成功
SELECT (SELECT COUNT(*) FROM players), (SELECT COUNT(*) FROM items), (SELECT COUNT(*) FROM inventory);
```

制約

おかしなデータを、入る前に止める

今の players には、名前が空の人も、レベル −5 の人も、ジェムがマイナスの人も入れられてしまう。

制約	止めるもの	ゲームでの例
PRIMARY KEY	重複と空（行を見分けられない）	同じ id のプレイヤーが2人
NOT NULL	空（NULL）	名前のないプレイヤー
UNIQUE	重複	同じログイン ID が2人
CHECK (条件)	条件に合わない値	レベル 0、ジェムがマイナス
DEFAULT 値	（止めない）省略時に入る値	新規登録でレベル 1、ジェム 0
FOREIGN KEY	存在しない相手を指す番号	いないプレイヤーの持ち物

POINT　プログラムのバグや不正な通信があっても、**データベースが最後の砦**になって壊れたデータを拒否する。C# 側のチェックと二重に守る。

制約

制約つきのテーブルを作って、わざと失敗させる

```
CREATE TABLE accounts (
id INTEGER PRIMARY KEY,
login_id TEXT NOT NULL UNIQUE,
name TEXT NOT NULL,
level INTEGER NOT NULL DEFAULT 1 CHECK (level BETWEEN 1 AND 99),
gems INTEGER NOT NULL DEFAULT 0 CHECK (gems ⩾ 0)
);
```

1行ずつ実行する SQL	結果
INSERT INTO accounts (login_id, name) VALUES ('allen01', 'アレン');	成功（level と gems は初期値）
INSERT INTO accounts (login_id, name) VALUES ('allen01', 'アレン2');	UNIQUE constraint failed
INSERT INTO accounts (login_id) VALUES ('mina01');	NOT NULL constraint failed
INSERT INTO accounts (login_id, name, level) VALUES ('go01', 'ゴウ', 0);	CHECK constraint failed
UPDATE accounts SET gems = gems - 100 WHERE id = 1;	CHECK constraint failed

演習 3-1

ショップの商品テーブル：成功か失敗か予想しよう

```
CREATE TABLE shop_items (
id INTEGER PRIMARY KEY,
name TEXT NOT NULL UNIQUE,
price INTEGER NOT NULL CHECK (price > 0),
stock INTEGER NOT NULL DEFAULT 0 CHECK (stock ⩾ 0)
);
```

No.	上から順に1つずつ実行する SQL	予想
a	INSERT INTO shop_items (name, price) VALUES ('ポーション', 50);	成功 ／ 失敗
b	INSERT INTO shop_items (name, price) VALUES ('ポーション', 80);	成功 ／ 失敗
c	INSERT INTO shop_items (name, price) VALUES ('エーテル', 0);	成功 ／ 失敗
d	INSERT INTO shop_items (name) VALUES ('エリクサー');	成功 ／ 失敗
e	UPDATE shop_items SET stock = stock - 1 WHERE name = 'ポーション';	成功 ／ 失敗
f	UPDATE shop_items SET stock = 10 WHERE name = 'ポーション';	成功 ／ 失敗

演習 3-1 解答

結果と、止めた制約

No.	結果	エラー	理由
a	成功	—	stock は書いていないので DEFAULT の 0 が入る
b	失敗	UNIQUE constraint failed	同じ名前のポーションがすでにある
c	失敗	CHECK constraint failed	値段 0 は price > 0 に合わない
d	失敗	NOT NULL constraint failed	price を書いていないので NULL になる
e	失敗	CHECK constraint failed	在庫 0 から 1 引くと −1 になる
f	成功	—	在庫を 10 に補充

POINT e は「在庫がないのに売ってしまう」バグを、データベースが止めた例。制約は**テーブルを作るときに決める**のが一番楽。

外部キー

存在しない相手を指す番号を止める

```
PRAGMA foreign_keys = ON; -- 外部キーのチェックを有効にする
-- ガチャで何が出たかの記録
CREATE TABLE gacha_logs (
id INTEGER PRIMARY KEY,
player_id INTEGER NOT NULL,
item_id INTEGER NOT NULL,
created_at TEXT DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (player_id) REFERENCES players(id),
FOREIGN KEY (item_id) REFERENCES items(id)
);
```

```
-- id 99 のプレイヤーはいない
INSERT INTO gacha_logs (player_id, item_id) VALUES (99, 1);
→ FOREIGN KEY constraint failed
```

- **FOREIGN KEY (列) REFERENCES 表(列)**：この列は、あの表の id のどれかでなければならない
- 指されている側（players）の行を消そうとしても止まる
- CURRENT_TIMESTAMP：入れた日時が自動で記録される

注意 SQLite は初期設定では外部キーをチェックしない。**PRAGMA foreign_keys = ON;** を先に実行しておく。ページを再読み込みしたら、また実行し直す。

トランザクション

ガチャの途中でサーバーが止まったら？

ガチャを1回引くと、サーバーの中では**3つの SQL** が順番に実行される。

1
ジェムを 300 減らす
`UPDATE players …`

ここで停電

2
アイテムを渡す
`INSERT INTO inventory …`

3
履歴を残す
`INSERT INTO gacha_logs …`

ジェムだけ減って、アイテムはもらえない。
「お金を払ったのに何も出なかった」という最悪の問い合わせになる。

POINT 欲しいのは「**3つ全部成功する**か、**1つも起きなかったことにする**か」のどちらか。途中の状態を絶対に残さない仕組みが**トランザクション**。

BEGIN で始め、COMMIT で確定する

```
BEGIN;
UPDATE players SET gems = gems - 300 WHERE id = 1;
INSERT INTO inventory (player_id, item_id, count) VALUES (1, 4, 1);
INSERT INTO gacha_logs (player_id, item_id) VALUES (1, 4);
COMMIT; -- 問題があれば ROLLBACK;
```

命令	意味
BEGIN	ここからひとまとまり
COMMIT	全部を確定する
ROLLBACK	BEGIN の前に全部戻す

- COMMIT する前に止まったら、データベースは**自動で BEGIN の前に戻す**
- ジェムが足りない、在庫がない、などをプログラムが見つけたら、自分で ROLLBACK する
- 試すときは COMMIT の代わりに ROLLBACK; を実行し、SELECT で何も変わっていないことを確かめる

演習 3-2

トランザクションで書いてみよう

No.	問題
1	ガチャ：ミナ（id 2）がジェムを 300 使い、鉄の剣（id 2）を引いた。ジェムを減らし、持ち物に加え、ガチャ履歴を残す
2	取り消し：ゴウ（id 3）のジェムを 300 減らしたところで、残高が足りないことに気づいた。SELECT で確かめてから、なかったことにする
3	トレード：ソラ（id 7）が持っているやくそう2個のうち1個を、ケン（id 5）に渡す

ヒント

形は **BEGIN; → 書き換えの SQL をいくつか → COMMIT;**。やめるときは COMMIT の代わりに ROLLBACK;。3 は「減らす」と「増やす（ケンは持っていないので追加）」の2つ。

演習 3-2 解答

答えと結果

```
-- 1 → ミナのジェムは 1200 から 900 に
BEGIN;
UPDATE players SET gems = gems - 300 WHERE id = 2;
INSERT INTO inventory (player_id, item_id, count) VALUES (2, 2, 1);
INSERT INTO gacha_logs (player_id, item_id) VALUES (2, 2);
COMMIT;
-- 2 → 途中の SELECT では -300、ROLLBACK のあとは 0 に戻る
BEGIN;
UPDATE players SET gems = gems - 300 WHERE id = 3;
SELECT gems FROM players WHERE id = 3;
ROLLBACK;
-- 3 → ソラのやくそう 1個、ケンのやくそう 1個
BEGIN;
UPDATE inventory SET count = count - 1 WHERE player_id = 7 AND item_id = 1;
INSERT INTO inventory (player_id, item_id, count) VALUES (5, 1, 1);
COMMIT;
```

こんなものもある

速くする仕組みと、狙われる仕組み

インデックス（効率化）

本の巻末の**索引**のようなもの。100万人の中から探すときも、全員を見ずに一瞬で見つかる。ランキングが速く出るのもこのおかげ。

```
CREATE INDEX idx_players_score
ON players (score);
```

付けすぎると、書き込みが遅くなる。

SQL インジェクション（攻撃）

名前の入力欄に **' OR 1=1 --** のような文字を入れて、SQL の意味を書き換える攻撃。他人のデータを丸ごと見られてしまう。

防ぎ方：入力された文字を、**SQL の文字列に直接つなげない**。来週、C# での正しい書き方を学ぶ。

POINT 今は名前を知っておけば十分。どちらも、実際にサーバーを作るときに必ず出てくる。

総合演習

ゲームの画面ごとに、サーバーが実行する SQL を書こう

準備 まず **setup_final.sql** を実行して、5つのテーブルを最初の状態にそろえる。

画面	必要なデータ
1 ランキング	スコア上位5人の名前・職業・スコア
2 マイページ	ミナ（id 2）の名前・レベル・ジェムと、持ち物の名前・レア度・個数（2つの SQL に分けてよい）
3 アイテム図鑑	すべてのアイテムの名前と、それを持っているプレイヤーの人数（0人のアイテムも出す）
4 新規登録	accounts に、ログイン ID 'nagi01'、名前「ナギ」で登録する（レベルとジェムは初期値）
5 ガチャ	ユイ（id 6）がジェムを 500 使い、ドラゴンの盾（id 4）を引いた。トランザクションで処理する

POINT 来週は、この SQL を C# のサーバーに組み込み、Unity の画面から呼び出す。**ここで書いた SQL が、そのまま来週の部品になる**ので、答えをノートに残しておく。

答えと結果

```
-- 1 ランキング → ユイ、ミナ、ハル、アレン、ゴウ
SELECT name, job, score FROM players ORDER BY score DESC LIMIT 5;
-- 2 マイページ → ミナ 47 1200 ／ やくそう N 3、魔法の杖 R 1
SELECT name, level, gems FROM players WHERE id = 2;
SELECT i.name, i.rarity, inv.count FROM inventory AS inv
JOIN items AS i ON inv.item_id = i.id WHERE inv.player_id = 2;
-- 3 アイテム図鑑 → やくそう 4、鉄の剣 2、魔法の杖 2、ドラゴンの盾 1、伝説の剣 2
SELECT i.name, COUNT(inv.player_id) AS owners FROM items AS i
LEFT JOIN inventory AS inv ON i.id = inv.item_id GROUP BY i.id, i.name;
-- 4 新規登録
INSERT INTO accounts (login_id, name) VALUES ('nagi01', 'ナギ');
-- 5 ガチャ → ユイのジェムは 2500 から 2000 に
BEGIN;
UPDATE players SET gems = gems - 500 WHERE id = 6;
INSERT INTO inventory (player_id, item_id, count) VALUES (6, 4, 1);
INSERT INTO gacha_logs (player_id, item_id) VALUES (6, 4);
COMMIT;
```

総合演習（チャレンジ）

もう一歩先の画面を作ろう

No.	問題（総合演習の続きのデータで解く）
1	職業別ランキング：剣士の中で、スコアが一番高いプレイヤーの名前とスコア
2	資産ランキング：全員の持ち物の合計金額（何もなければ0）を、高い順に
3	平均より上：スコアが全体の平均より高いプレイヤーの名前とスコア
4	職業の人気：職業ごとの人数とジェムの合計を、人数の多い順に

ヒント　2 は LEFT JOIN を2回つなぐ。3 は WHERE の中に SELECT を書ける：**WHERE score > (SELECT AVG(score) FROM players)**。

答えと結果

```
-- 1 → ハル 99000
SELECT name, score FROM players WHERE job = '剣士' ORDER BY score DESC LIMIT 1;
-- 2 → ユイ 6550、アレン 5350、ゴウ 1500、ミナ 380、リサ 100、ソラ 20、ケン 0、ハル 0
SELECT p.name, COALESCE(SUM(i.price * inv.count), 0) AS assets
FROM players AS p
LEFT JOIN inventory AS inv ON p.id = inv.player_id
LEFT JOIN items AS i ON inv.item_id = i.id
GROUP BY p.id, p.name
ORDER BY assets DESC;
-- 3 → アレン、ミナ、ゴウ、ユイ、ハル（平均は 83562.5）
SELECT name, score FROM players
WHERE score > (SELECT AVG(score) FROM players);
-- 4 → 剣士 3 / 1100、魔法使い 2 / 3200、ほかは 1
SELECT job, COUNT(*) AS member_count, SUM(gems) AS total_gems
FROM players GROUP BY job ORDER BY member_count DESC;
```